

Predicting What a Medicine Will Do, Before Making It

A plain-English guide to this project: what it does, how it works, what it is built with, and what we found. No science background needed.

1. The one-paragraph version

Making a new medicine is slow and expensive. Before a chemist makes a substance in the lab, it helps to guess whether it will be useful or harmful. This project builds computer programs that look at the **recipe of a molecule** and predict things like "is this toxic?" or "will this dissolve in water?". **But the real point of the project is not the predictor. It is the measuring.** We built a very strict way of testing whether such programs actually work, and then used it on our own program. It turned out that our fancy program was no better than a very simple one. We are reporting that honestly, because most papers in this field never check carefully enough to find out.

2. Key words, one line each

Everything in this project is built out of these ideas.

Word	What it means in one line
Molecule	A substance, like caffeine or aspirin, made of atoms joined together.
Property	Something we want to know about it, like "is it poisonous?" or "does it dissolve?".
Model	A computer program that learns patterns from examples and then makes guesses.
Training	Showing the program thousands of known examples so it can learn the pattern.
Dataset	A big table of molecules where the answer is already known, used for training and testing.
Prediction	The program's guess for a molecule it has never seen before.
Feature / representation	A way of turning a molecule into numbers, because computers only handle numbers.
Baseline	A simple method we compare against, to check whether the complicated one is worth it.
AUC	A score from 0.5 to 1.0 for yes/no questions. 0.5 is random guessing, 1.0 is perfect.
RMSE	An error score for number questions. Lower is better. 0 would be perfect.

3. What problem are we solving?

A chemist can draw a molecule on paper long before making it in the lab. Making it costs money and months of work. So the question is:

Given only the drawing of a molecule, can a computer predict what it will do?

If yes, chemists can test thousands of ideas on a laptop overnight and only make the most promising few in the lab. That is why this field exists.

4. The data we use

We use **8 public datasets** called MoleculeNet. They are the standard test sets in this field, so our results can be compared with other people's. Each one is a table: one row per molecule, with the correct answer already filled in by real lab experiments.

Dataset	How many molecules	What it asks	Type
Tox21	7,823	Is this toxic? (12 different toxicity tests)	Yes / No
BBBP	2,039	Can this reach the brain through the blood?	Yes / No
ClinTox	1,480	Did this fail safety trials in humans?	Yes / No
BACE	1,513	Does this block an enzyme linked to Alzheimer's?	Yes / No
SIDER	1,427	What side effects does this drug have? (27 kinds)	Yes / No
ESOL	1,128	How well does this dissolve in water?	A number
Lipophilicity	4,200	Does this prefer fat or water?	A number
FreeSolv	642	How much energy to dissolve it in water?	A number

Five datasets ask **yes/no** questions (scored with AUC) and three ask for **a number** (scored with RMSE). We deliberately left out a ninth dataset, HIV, because it is huge and would have cost about 60 hours of computing without making our conclusions any stronger.

5. How a molecule becomes numbers

A computer cannot read a drawing of a molecule. So we describe the same molecule in **five different ways**. Each way notices something the others miss. We call each one a **view**, like photographing the same object from five angles.

View	How it describes the molecule	What it is good at
Graph (GIN)	As a network: atoms are dots, bonds are lines joining them.	The shape and how atoms connect.
Graph (GINE)	The same, but it also notices what <i>kind</i> of bond each line is.	A little more detail than GIN.
AttentiveFP	Also a network, but it learns which atoms to pay most attention to.	A well-known published method; we use it as a fair comparison.
Sequence (ChemBERTa)	As text. Chemists write molecules as strings like "CCO" for alcohol.	Patterns in how chemists write molecules.
Descriptor / fingerprint	As a long list of measured facts: weight, size, oiliness, and which small fragments appear.	Known chemistry, calculated directly.

ChemBERTa is worth one extra line: it is a language model, like a tiny cousin of ChatGPT, but it was trained on millions of molecule strings instead of English sentences. It already knows a lot about chemistry before we start.

6. The architecture (how the program is built)

The whole system is a pipeline. A molecule goes in one end and a prediction comes out the other.



Stage	What happens here, in plain words
1. Molecule	One substance, written as a text string by a chemist.
2. Five views	We describe it five different ways at once (previous page). Each view produces its own list of numbers.
3. Make same size	The five lists come out different lengths, so we shrink or stretch each one to exactly 256 numbers. This is important for fairness, see below.
4. Fusion	The step that combines the five descriptions into one. This was meant to be the clever part of the project.
5. Head	A small final calculator that turns the combined description into the actual answer.
6. Prediction	"87% likely to be toxic", or "dissolves at -3.2".

Why step 3 matters (a mistake we caught)

At first, each view fed a final calculator sized to match it. The text view produced a big list, so it got a big calculator; others got small ones. That meant when one view won, we could not tell if it was because the **description** was better or just because its **calculator was bigger**. We fixed this by forcing every view to exactly 256 numbers and giving every view the same size calculator. Then we re-ran everything. This is the kind of unfairness that is easy to miss and changes results.

7. The five ways of combining (the "ladder")

We did not build just one combiner. We built five, from simplest to most complex, so we could see **which part actually helps**. If only the complicated one had been built, we would never have known that the simple one works just as well.

Name	What it does	Size (settings it learns)
concat	Just glues the five descriptions end to end. No thinking.	0
gated	Learns how much to trust each view, per molecule. Like a volume knob per view.	16,513
xattn	Lets the views "talk to each other" and compare notes.	1,054,208
bilinear	Multiplies views together to catch combined effects.	99,072
proposed	All three of the above, stacked. Our most complex design.	1,169,793

The last column is roughly "how many dials the program can adjust". The **proposed** combiner has about **71 times more dials** than the simple **gated** one. Remember that number for the results section.

8. How the data is split (very important)

You cannot test a student using the exact questions they revised from. Same for a computer program. So we cut each dataset into three parts:

Part	Share	What it is used for	Can the model see the answers?
Train	80%	Learning the patterns.	Yes, this is the textbook.
Validation	10%	Deciding when to stop learning, and picking settings.	Yes, this is the mock exam.
Test	10%	The final score we report.	No. Touched once, at the very end.

Why we split by "scaffold" and not randomly

A **scaffold** is the core skeleton of a molecule. Many molecules share one skeleton with small bits changed.

- If we split **randomly**, near-identical molecules land in both train and test. The program is basically being tested on what it memorised, and scores look great but are fake.
- If we split **by scaffold**, all molecules sharing a skeleton go to the same side. Now the test contains genuinely *new* shapes. Scores drop, but they are honest.
- This matches real life: a chemist wants to predict something **new**, not something already in the books.

Why we do this six times, not once

One split is one roll of the dice. Do it a different way and the score changes. We found the same model, on the same data, scoring **0.68 under one split and 0.90 under another** — a gap of 0.22, which is larger than almost any improvement papers claim. So we use **6 different splits** (1 standard + 5 shuffled) and report the **average with a range**. A result that only shows up in one split is not a result.

9. How training actually works

Training is repetition with correction. Here is one full cycle, called an **epoch**:

Step	What happens
1	Take a small batch of 128 molecules from the train part.
2	The program guesses the answer for each one.
3	Compare guesses with the real answers and measure how wrong it was. This number is the loss .
4	Nudge all the internal dials slightly in the direction that would have been less wrong.
5	Repeat for the next batch, until all training molecules have been seen once. That is one epoch.
6	Now test on the validation part. Did it improve?
7	If it keeps improving, run another epoch. If it stops improving for 15 epochs in a row, stop .
8	Go back to the best epoch, and only now run the test part. That score is reported.

Step 7 is called **early stopping**. Without it the program starts memorising the training molecules instead of learning general patterns, which is called **overfitting** — like a student who memorises answers but cannot handle a new question.

One more rule we follow: some molecules have **missing** answers, because nobody ran that lab test. We skip those cells. The original code treated missing as "not toxic", which invented fake answers for **24% of the Tox21 test data**. Finding and fixing that changed the results.

10. Technology stack

Everything is written in **Python**. Each tool below does one job.

Tool	Version	What it does for us, in one line
Python	3.11	The programming language everything is written in.
PyTorch	2.3.1	The main engine for building and training the models.
PyTorch Geometric	2.6.1	An add-on that lets PyTorch handle molecules-as-networks.
RDKit	2025.3.5	The chemistry toolkit. Reads molecule strings, calculates chemical facts.
DeepChem	2.8.0	Supplies the 8 standard datasets and the standard scaffold split.
Transformers	4.43.4	Runs ChemBERTa, the molecule-reading language model.
PEFT (LoRA)	0.12.0	Lets us fine-tune the big language model cheaply, by training only a tiny part of it.
scikit-learn	1.4.2	Classic machine learning: Random Forest, and the scoring functions.
SciPy / NumPy / pandas	—	Maths, statistics and table handling.
Chemprop	2.3.1	An outside published model, used as an independent yardstick.
Optuna	4.5.0	A settings-tuner. Written into the project but deliberately never run — see Q8.
Git & GitHub	—	Keeps every version of the code and results, so nothing is lost.
Google Colab	—	Free cloud computers with GPUs, for the long training runs.

A **GPU** is a chip originally made for video games. It does thousands of small sums at once, which is exactly what training needs. Our biggest runs took about 11 hours on one.

11. What we found

These are the real results, stated plainly.

Question we asked	Honest answer
Does our combined model beat the old system we started from?	Yes. It won on all 8 datasets out of 8. This is a genuine, solid win.
Does it beat a simple list of chemical facts (the descriptor view)?	No. It won on 4 of 8 — that is a coin flip. The simple method is just as good.
Is the complex combiner better than the cheap one?	No. 71 times more dials bought us nothing that survives careful checking.
Do we need the molecule-as-network view at all?	No. Removing it changed nothing, ran 13 times faster , and used 59% fewer dials.
Does a famous published model (AttentiveFP) beat the old simple one?	No , not reliably — at 10 times the size. So this is not just our model being weak.
Does training on a different computer give the same answer?	No. Same code, same settings, different GPU: 18% of single-split scores moved more than the size of effects people publish.
The honest headline: our model beats the thing it replaced, and loses to a simple list of chemical facts. Both halves are true and we report both. Many papers would report only the first half.	

12. The "how sure are you?" part

A prediction of "toxic" is not much use without knowing how confident the program is. So we added a method called **conformal prediction**.

- Instead of one answer, it gives a **set** of answers with a promise: "the truth is in here 90% of the time".
- If the program is unsure it returns both "toxic" and "safe", which is its way of saying *I do not know*. That is more useful than a confident wrong answer.

We found a serious problem with the standard version:

It keeps its 90% promise **overall** — but only because most molecules are safe. For the **toxic** ones, the ones you actually care about, it was right as little as **10% of the time** in the worst case. The promise was technically true and practically useless. A fix called **class-conditional** conformal prediction repairs it, by making a separate promise for each answer.

Be ready for this one: another team published this same discovery in July 2026, four months before us. We found their paper, we cite it, and we now present our version as **independent confirmation on more models** rather than as our discovery. Saying this openly is the correct thing to do.

13. Questions a teacher may ask

Q1. What is actually new here, if the model did not win?

The strict testing method, and what it revealed. We found 6 real errors in the system we inherited, and we proved that two popular ideas in recent papers do not help. A carefully proven "this does not work" is useful, because it saves other people from repeating it.

Q2. Why is a negative result worth anything?

Because we can show it is trustworthy. We tested on 8 datasets, 6 splits each, with statistical correction. Most papers use 1 or 2 datasets and 1 split, where luck can look like success.

Q3. How do you know your code is not simply broken?

We tested the parts separately. We checked the combining step really does mix the views, by proving the simplest one is exactly additive and the complex one is not. We also checked our uncertainty method on made-up data with known answers first, and it was correct there.

Q4. Why 8 datasets, why not 3?

Statistical strength comes from the number of datasets. With 8, the best possible confidence score is 0.0078, which is strong. With 5 it could never reach the usual 0.05 threshold at all.

Q5. What is overfitting, and how did you prevent it?

It is memorising instead of learning. We prevented it with early stopping, by splitting by scaffold so the test is genuinely new, and by never letting the model see the test data until the very end.

Q6. What is data leakage? Did you have any?

It is when answers sneak from the test side into training, making scores fake. The inherited system had it: it was picking its best model by looking at the test scores. We fixed that so all choices are made on the validation part only.

Q7. Why did you use a language model on chemistry?

Chemists already write molecules as text strings. A language model trained on millions of those strings learns chemical patterns the same way a text model learns grammar.

Q8. Did you tune the settings to get the best scores?

Deliberately not. Every model uses one identical setting. If we tuned only our model and compared it to untuned ones, we would be measuring effort, not quality. We wrote the tuning tool and left it switched off on purpose, and we state this as a limitation.

Q9. What is the single most useful thing you found?

That the same code with the same random seed, run on a different GPU, gives different results — enough to change 18% of single-split scores by more than the effects papers usually report. Averaging over 6 splits absorbs it. A single-number result does not.

Q10. What would you do with more time?

Run the settings-tuner fairly on every model, add more outside comparison models, and test on harder real-world data rather than public benchmark sets.

Q11. Can this be used in a real company?

The predictor is not better than simple chemistry, so not as a product yet. The testing method is usable immediately — any team can run it to check whether their own model is really working.

Q12. How big is the project?

About 50 commits of work, 8 datasets, 6 splits, 13 models compared, roughly 30 hours of training time across laptop and cloud GPUs, and an automatic checker that re-verifies every number in the write-up against the saved results.

If you remember one sentence: we built a fair way to test molecule-prediction programs, used it on our own, and it honestly told us our complicated idea was no better than a simple one — which is exactly what a good measuring tool is supposed to do.